EPITECH · 2D GAME · JAVA & POO

Fakemon Arena

Plan de projet, méthode Scrum et répartition des tâches entre Dev A et Dev B

13 jours
ouvrés, du mardi 3 au vendredi 20 ; rendu au plus tard dimanche 22 au soir

4 sprints
de 2 à 4 jours, avec planning, dailies, review et rétro

35 issues
31 planifiées + 4 bonus ; une issue = une branche = une PR

36 / 36
points planifiés pour A / B, sur 39 de capacité chacun

Les 20 jours en un coup d'œil



S0 Conception · **S1** Fondations · **S2** Moteur & dashboard · **S3** Intégration & release · week-ends et 11 novembre non planifiés

A Dev A · cœur du jeu & données

Tout ce qui décide : modèle, moteur de combat, talents, objets, IA, données JSON, roster des 30 fakemon, CI.

Rôle Scrum : Scrum Master

B Dev B · expérience joueur & présentation

Tout ce que le joueur voit et entend : écrans JavaFX, dashboard d'ajout de fakemon, animations, sons, sprites, soutenance. Le LAN en bonus.

Rôle Scrum : Product Owner

Fakemon Arena est un nom de travail. Hypothèses du plan : 6 h effectives par jour ouvré et par personne ; 1 point ≈ 2 h de travail pour vous, apprentissage compris, soit 3 points par jour. Les week-ends et le 11 novembre ne sont pas planifiés : ils servent uniquement de marge.

1 Le jeu en bref

Un jeu de combat de monstres au tour par tour, en 6 contre 6. Chaque joueur reçoit une équipe tirée au hasard dans un roster de 30 fakemon originaux, puis affronte l'IA.

Format	Combat simple 6 contre 6. Chaque joueur reçoit 6 fakemon tirés sans doublon dans le roster (30 de base + ceux ajoutés via le dashboard), avec 4 attaques tirées parmi leurs attaques possibles, un talent et un objet tenu.
Modes	Joueur contre IA. Combat en LAN entre deux PC : bonus, seulement avec du temps libéré (Go / No-Go le lundi 16).
Mécaniques	10 types originaux, bonus de même type, coups critiques, précision, priorité, vitesse, switchs, KO, talents passifs, objets tenus. Statuts en bonus.
Habillage	JavaFX, sprites pixel art 32×32 affichés ×3 (le dos est la face retournée, sauf si un vrai dos est fourni), animations, musiques, effets sonores, un cri par fakemon.
Outil	Dashboard d'ajout de fakemon : sprite avant, sprite arrière, cri, attaques disponibles, nom, nom français (détail en section 6).
Stack	Java 21, JavaFX 21, Maven, JUnit 5, JaCoCo, Gson, GitHub (Issues, Projects, Actions), PlantUML dans `docs/uml`.

> **Options validées**
>
> JavaFX · adversaire IA, LAN en bonus conditionnel · talents et objets tenus · animations et sons · dashboard d'ajout de fakemon · 6 h par jour ouvré · week-ends libres.

2 Ce que le sujet exige, et où c'est traité

Exigence du sujet	Traduction dans le projet	Issues	Dev
Jeu en Java respectant les principes POO	Hiérarchies polymorphes : actions, events, talents, objets, IA. Aucune logique dupliquée.	#4, #8, #11, #24	A
Interface graphique du jeu et des personnages	Écrans JavaFX : menu, aperçu des équipes, combat, panneau d'équipe, fin, dashboard.	#12–15, #19, #20, #26	B
Game Design Document complet avant le code	Sprint 0 dédié. Aucune feature codée avant la validation du GDD (mercredi 4).	#2, #3	A B
Diagrammes UML avec les design patterns	v1 avant de coder, version finale au Sprint 3. Chacun dessine ses notions.	#5, #6, #25, #30	A B
Tests unitaires de la logique (combat, objets, victoire et défaite)	Écrits dans chaque issue du cœur (Definition of Done), aléatoire injectable pour des tests reproductibles.	#9, #17, #18, #24	A
Exécution automatisée des tests	GitHub Actions sur chaque PR et sur `main` : une PR rouge ne peut pas être mergée.	#7	A
Couverture de tests	Rapport JaCoCo à chaque PR, objectif de 70 % sur le cœur.	#7, #25	A
Documentation complète (Javadoc)	Exigée par la Definition of Done, relue en fin de projet.	#25, #30	A B
Soutenance : documents, conception, démo jouable	Support, script de démo, répétition le vendredi 20.	#31	B

3 Partir de votre niveau : la piscine Java

Le plan s'appuie sur ce que vous avez pratiqué pendant les 10 jours de piscine et n'ajoute que le strict nécessaire. Chaque techno nouvelle s'apprend dans l'issue qui en a besoin, et son estimation inclut ce temps d'apprentissage.

Déjà vu en piscine, et où ça sert

Jour	Notions	Où vous les réutilisez
J02 – J03	Classes, constructeurs, visibilité, `static`, packages	Le modèle (`Species`, `Fakemon`, `Team`) et un package par notion
J04	`enum`, héritage, `@Override`, `instanceof`	`Type` en enum comme `Animal.Type` ; sous-classes d'actions et d'events
J05	Classes abstraites, interfaces (`Fighter`, `Unit`, `SpaceArena`)	`SpaceArena` était déjà un petit moteur de combat au tour par tour : `BattleEngine` en est la version complète
J06	Exceptions personnalisées, `final`	`InvalidSpeciesException` pour le dashboard, équipe pleine
J07	Génériques, `Comparable`	Listes typées, tri des actions par priorité puis vitesse
J08	Factory, Observer, Decorator, UML, SOLID	Fakemon créés depuis le JSON ; events du moteur vers l'écran ; objets tenus comme `Shield` et `FireSword`
J09	Annotations, Javadoc, mini-framework de test	JUnit 5 fonctionne comme votre `TestRunner` (`@Test`, `@BeforeEach`) ; Javadoc
J10	`Map`, exceptions, génériques bornés	Registre des espèces et des attaques

Nouveau pour vous, et quand l'apprendre

Techno	Point de départ	Issue	Dev
Maven + JavaFX	Le bootstrap JavaFX fourni (projet VS Code, fenêtre, `Timeline`)	#1, #12, #13	B
JUnit 5 + JaCoCo	Même logique que votre `TestRunner` du J09	#7, #8	A
GitHub Actions	Le modèle « Java with Maven » proposé par GitHub	#7	A
Gson + fichiers	`java.nio.file.Files` et `new Gson().fromJson(...)`	#22	A
PlantUML	Diagrammes écrits en texte, rendus en PNG	#5, #6	A B
Son et animations JavaFX	`AudioClip`, `FileChooser`, transitions	#20, #27, #28	B
Threads et sockets	Pas vu en piscine : c'est pour ça que le LAN est un bonus	#34, #35	B

Ce que le plan simplifie en conséquence

- **Aucun thread dans le jeu** : l'écran appelle `engine.playTurn(...)`, le moteur prévient l'écran par des events (Observer). Pas d'attente, pas de synchronisation.
- **Que des classes et interfaces classiques**, comme en piscine : pas de records ni de classes scellées.
- **Patterns limités** à ceux du J08, plus Strategy et Template Method, qui reprennent l'idée de `Fighter` et `Unit` au J05.
- **Assets légers** : sprites 32×32 de face seulement, cris générés automatiquement avec jsfxr.
- **Périmètre trié** : P0 indispensable, P1 qui saute en premier en cas de retard, P2 en bonus.

4 Répartition : une notion, un propriétaire

Chaque notion du projet appartient à un seul dev, qui la conçoit, la code, la teste, la documente et la présente. La frontière entre les deux est le contrat `core.api` : A écrit tout ce qui décide, B tout ce que le joueur voit et entend.

A Dev A · cœur du jeu & données

Notion (propriétaire unique)	Issues	Pts
Modèle & moteur de combat Espèces, fakemon, équipes ; types, dégâts, tours, switchs, KO, tirage des équipes.	#8, #9, #10, #17, #18	15
Talents & objets Talents appelés par le moteur, objets tenus en Decorator. Statuts en bonus.	#24 bonus #32	3 +3
IA adverse IA aléatoire. IA heuristique en bonus.	#11 bonus #33	1 +2
Données & roster Lecture et écriture JSON, validation, dossier utilisateur, les 30 fakemon.	#22, #23	7
Qualité CI GitHub Actions et rapport de couverture.	#7	2
Conception du cœur GDD (règles, contenu), contrat d'API, UML du cœur, Javadoc.	#3, #4, #6, #25	8
Total Dev A (hors bonus)		**36**

B Dev B · expérience joueur & présentation

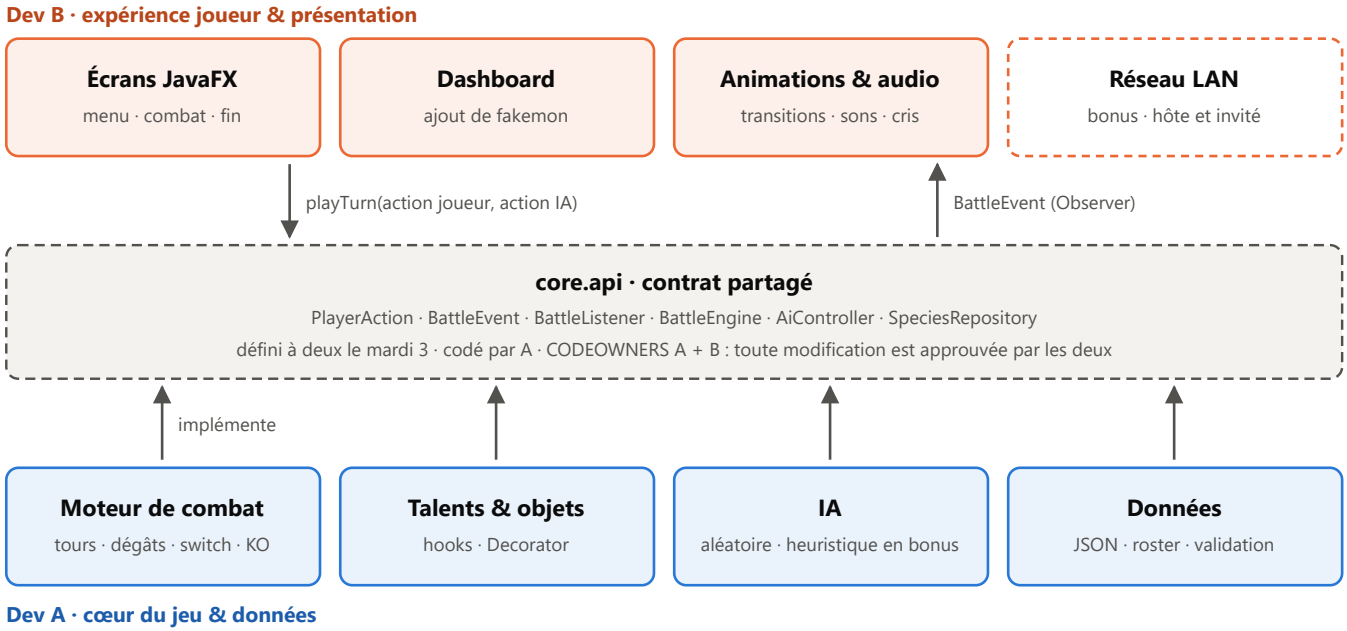
Notion (propriétaire unique)	Issues	Pts
Écrans JavaFX Squelette, navigation, menu, aperçu, combat, panneau d'équipe, fin, branchement, bannières.	#1, #12, #13, #14, #15, #19, #26, #29	16
Dashboard d'ajout de fakemon Formulaire complet, aperçu des sprites, écoute du cri, choix des attaques.	#20	5
Animations & audio Transitions de combat, musiques, effets, cris.	#27, #28	4
Direction artistique Charte, 30 sprites, icônes des types.	#16, #21	3
Réseau LAN (bonus) Spike puis combat entre deux PC, seulement avec du temps libéré.	bonus #34, #35	— +6
Conception de l'expérience & présentation GDD (vision, UX, DA), UML des écrans, README, soutenance.	#2, #5, #30, #31	8
Total Dev B (hors bonus)		**36**

Règles qui gardent la répartition saine

- **Relecture croisée systématique** : A relit toutes les PR de B et inversement. Chacun garde ses notions, mais connaît le code de l'autre et peut le présenter en soutenance.
- **Pas d'attente entre les deux** : B construit les écrans sur des events factices créés avec les classes du contrat (#4), puis les branche sur le vrai moteur le mardi 17 (#26).
- **Frontière du dashboard** : B construit l'écran (#20), A fournit la sauvegarde et la validation (#22). B affiche les erreurs renvoyées par A sans les recoder.
- **Équilibre de charge** : 36 points planifiés chacun. Par sprint (planifié / capacité), A : S0 5/6 · S1 11/12 · S2 8/9 · S3 12/12 ; B : S0 5/6 · S1 11/12 · S2 8/9 · S3 12/12.

5 Architecture et contrat d'interface

Le contrat est défini à deux le premier jour, pendant un atelier de 2 h : ce sont les seules classes que les deux devs touchent. Ensuite, chacun avance en parallèle de son côté de la frontière.



Un tour de jeu, sans thread

1. Le joueur clique sur une attaque ou un fakemon : l'écran crée un `UseMove` ou un `Switch`.
2. L'écran demande son choix à l'IA : `ai.chooseAction(engine.getState())`.
3. L'écran appelle `engine.playTurn(actionJoueur, actionIA)`.
4. Le moteur résout le tour et envoie chaque `BattleEvent` aux `BattleListener` abonnés (l'écran, le son).
5. L'écran affiche les events un par un, puis réactive les boutons. Si un fakemon est KO, il demande un remplaçant au joueur, puis appelle `engine.replace(...)`.

Design patterns à montrer dans l'UML

Pattern	Déjà vu	Où dans le projet	Dev
Observer	J08	`BattleListener` : le moteur publie, l'écran et le son s'abonnent	A B
Factory	J08	Création des espèces et des attaques depuis le JSON	A
Decorator	J08	Objets tenus qui modifient les stats, comme `Shield` et `FireSword`	A
Strategy	≈ `Fighter` (J05)	`AiController` : IA aléatoire, IA heuristique en bonus	A
Template Method	≈ `Unit` (J05)	`Move.execute()` : précision, puis dégâts, puis effet	A
State	nouveau	Statuts (bonus #32)	A
MVC	nouveau	Écrans JavaFX : vue, contrôleur, données affichées	B

Arborescence des packages

```
fr.epitech.fakemon
├── core.api      contrat partagé : actions, events, interfaces   A (CODEOWNERS : A + B)
├── core.model    Type, Stats, Species, Fakemon, Move, Team       A
├── core.engine   tours, dégâts, switchs, KO, tirage des équipes  A
├── core.effects  talents, objets tenus (statuts en bonus)        A
├── core.ai       IA aléatoire (heuristique en bonus)             A
├── data          JSON, SpeciesRepository, validation             A
├── ui            écrans, dashboard, animations, son              B
└── net           réseau LAN (bonus)                              B
resources/data/*.json → A    resources/sprites, audio, css → B
```

6 Dashboard d'ajout de fakemon

Un écran accessible depuis le menu principal pour créer un fakemon sans toucher au code. Le fakemon enregistré rejoint le pool de tirage au lancement suivant.

Champ	Contrôle JavaFX	Validation (règles de A)	Origine
Nom	Champ texte	Unique, 3 à 12 caractères	Demandé
Nom français	Champ texte	Unique, 3 à 12 caractères	Demandé
Sprite avant	`FileChooser` + aperçu	PNG carré, affiché ×3 sans lissage	Demandé
Sprite arrière	`FileChooser` + aperçu	Facultatif : sans lui, la face retournée est utilisée	Demandé
Cri (son)	`FileChooser` + bouton d'écoute	WAV, 2 secondes maximum	Demandé
Attaques disponibles	Liste filtrable par type, à cocher	Au moins 4 attaques	Demandé
Types	Deux listes déroulantes (la seconde est facultative)	1 ou 2 types distincts	Nécessaire au moteur
Stats de base	6 sélecteurs numériques + total affiché	PV, Attaque, Défense, Atq. Spé, Déf. Spé, Vitesse ; total plafonné à 600	Nécessaire au moteur

B Écran (#20, Sprint 2)

- Formulaire, sélecteurs de fichiers, aperçu des sprites, écoute du cri
- Liste des attaques à cocher
- Affichage des erreurs renvoyées par A

A Sauvegarde (#22, Sprint 3)

- `SpeciesRepository.save()` avec Gson
- Validation : `InvalidSpeciesException` (comme au J06)
- Fusion des fakemon ajoutés avec les 30 de base au chargement

Point technique à anticiper

Une fois le jeu packagé, les ressources du jar sont en lecture seule. Les fakemon créés par le dashboard sont donc écrits dans un dossier utilisateur (`~/.fakemon-arena/fakemon/<id>/` : `species.json`, `front.png`, `back.png`, `cry.wav`), que le repository fusionne avec le roster intégré.

> **Le dashboard sert d'abord à l'équipe**
>
> Jusqu'au Sprint 3, le dashboard enregistre en mémoire grâce à l'implémentation fournie avec le contrat (#4). Le jeudi 19, A saisit le roster des 30 fakemon avec le dashboard (#23) : c'est son premier vrai test. Si le dashboard n'est pas prêt, A écrit le JSON à la main et le dashboard le relira tel quel.

7 Organisation Scrum

Rôles

Rôle	Porté par	Responsabilités
Product Owner	B Dev B	Porte la vision joueur, ordonne le backlog, rédige les critères d'acceptation, accepte ou refuse les stories en review, décide quels P1 sautent en cas de retard, tague les releases.
Scrum Master	A Dev A	Anime les dailies et les rétros, tient le board et le burndown, lève les blocages, vérifie le respect de la Definition of Done.
Équipe de dev	A B	Les deux. Les rôles Scrum sont des casquettes en plus du développement, pas des postes à plein temps.

Sprints et capacité

Capacité = jours ouvrés × 3 points (6 h par jour, 1 point ≈ 2 h). Le P0 occupe 85 % de la capacité de A et 79 % de celle de B.

Sprint	Objectif de sprint	Capacité par dev	A planifié	B planifié	Tag
S0 Conception mar. 3 → mer. 4 nov. · 2 j	GDD validé, contrat d'API défini à deux, dépôt prêt. Aucune feature n'est codée avant la validation du GDD.	6	5 5 P0	5 5 P0	—
S1 Fondations jeu. 5 → mar. 10 nov. · 4 j	Modèle, types et dégâts testés ; la CI tourne ; menu, aperçu, combat et fin s'affichent sur des données factices.	12	11 11 P0	11 11 P0	v0.1.0
S2 Moteur & dashboard jeu. 12 → lun. 16 nov. · 3 j	Le moteur résout un combat complet en tests (switchs, KO, victoire) ; le dashboard permet de saisir un fakemon ; les 30 sprites existent.	9	8 8 P0	8 8 P0	v0.2.0
S3 Intégration & release mar. 17 → ven. 20 nov. · 4 j	Un combat 6 contre 6 jouable contre l'IA sur le vrai roster, avec talents, objets, animations et sons ; documentation et soutenance prêtes ; v1.0.0 vendredi 20.	12	12 9 P0 + 3 P1	12 7 P0 + 5 P1	v1.0.0
Total		**39**	**36**	**36**	

> **La marge, en quatre couches**
>
> 1. **Tampon dans chaque sprint** : aucun sprint n'est rempli de P0 jusqu'au bord ; le reste absorbe les imprévus et l'apprentissage.
> 2. **Les P1 sautent en premier** : A 3 points (≈ 6 h), B 5 points (≈ 10 h). Le PO décide lesquels à la review.
> 3. **Le 11 novembre et les deux premiers week-ends** ne sont pas planifiés : on ne s'en sert que pour rattraper un vrai retard.
> 4. **Le dernier week-end** : la v1.0.0 est visée pour le vendredi 20 ; samedi 21 et dimanche 22 ne servent qu'en cas de problème, avant la remise du dimanche soir.

Cérémonies

Cérémonie	Quand	Durée	Contenu	Anime
Sprint planning	1er jour du sprint, 9 h 30	30 min 1 h au S0	Objectif de sprint, choix des issues dans la limite de la capacité, découpage des grosses issues.	PO + SM
Daily	Chaque jour ouvré, 9 h 30 ; jamais le week-end	10 min	Hier, aujourd'hui, blocages, PR à relire. Mise à jour du board et du burndown.	SM
Affinage	Lundi 9 et mercredi 18	15 min	Préciser et réestimer les issues du sprint suivant, vérifier la Definition of Ready.	PO
Sprint review	Dernier jour du sprint, 17 h	30 min	Démo depuis `main` uniquement : ce qui n'est pas mergé n'est pas fini. Le PO accepte ou renvoie au backlog.	PO
Rétrospective	Juste après la review	30 min	Format différent à chaque sprint, 2 actions d'amélioration au maximum.	SM

```
Daily écrit (si l'un des deux est absent)
Hier        : #13 écran de combat, barres de PV terminées
Aujourd'hui : #13 boutons d'attaque + journal ; relire la PR #9
Blocage     : je ne sais pas afficher un sprite sans lissage, besoin de 10 min ensemble
```

Rétrospectives

Quand	Format	Pourquoi ce format
S0 · mer. 4	**Start / Stop / Continue**	Simple et rapide pour caler le fonctionnement du binôme.
S1 · mar. 10	**Mad / Sad / Glad**	Premier sprint de code, nouvelles technos : faire sortir les frustrations tôt.
S2 · lun. 16	**Voilier** : vent, ancres, rochers, île	Avant le dernier sprint : repérer les risques (rochers) qui restent.
Finale · ven. 20	**Frise du projet**	Courbe d'humeur jour par jour et 3 leçons pour le prochain projet.

Chaque action de rétro devient une issue étiquetée `retro-action` dans le sprint suivant, avec un responsable, et elle est vérifiée à la rétro d'après.

8 Calendrier

Les jours encadrés en haut portent un jalon. Les cases grisées ne sont pas planifiées : on ne les utilise que pour rattraper un retard.

lun.	mar.	mer.	jeu.	ven.	sam.	dim.
2 Veille du lancement	**3** S0 · J1 **Kick-off + planning** Atelier contrat d'API (2 h) Création des issues	**4** S0 · J2 **Validation du GDD** Review + rétro S0	**5** S1 · J1 **Planning S1** A : modèle · B : écrans	**6** S1 · J2 Daily	**7** Libre marge si retard	**8** Libre marge si retard
9 S1 · J3 Daily · affinage S2	**10** S1 · J4 **Review + rétro S1** tag v0.1.0	**11** Férié · libre marge si retard	**12** S2 · J1 **Planning S2** A : moteur · B : dashboard	**13** S2 · J2 Daily	**14** Libre marge si retard	**15** Libre marge si retard
16 S2 · J3 **Review + rétro S2** tag v0.2.0 **Go / No-Go LAN**	**17** S3 · J1 **Planning S3** Écran ↔ vrai moteur (#26)	**18** S3 · J2 Daily · affinage A : JSON (#22)	**19** S3 · J3 Roster saisi (#23) **Feature freeze 18 h**	**20** S3 · J4 **Review finale · v1.0.0** Répétition Rétro finale	**21** Libre marge si retard	**22** Libre dernier délai le soir

Go / No-Go du LAN · lundi 16 novembre, à la review du Sprint 2

Go seulement si tout le P0 des Sprints 1 et 2 est mergé et que B est à jour. Le LAN ne s'ajoute jamais à la charge : il utilise le temps libéré, et le PO peut lui sacrifier des P1 de B (#27, #28). Il se fait sur une branche à part, en 10 h maximum, et n'est mergé que si un combat complet passe le jeudi 19 à 18 h. **No-Go** : le LAN reste dans le backlog et la démo se fait contre l'IA, comme prévu.

Rendez-vous d'intégration

- **Mar. 3** : contrat d'API défini à deux (#4), base de tout le travail en parallèle.
- **Mer. 4** : GDD validé, feu vert pour coder.
- **Lun. 16** : moteur et switchs mergés (#17, #18), Go / No-Go du LAN.
- **Mar. 17** : l'écran de combat quitte les données factices et se branche sur le vrai moteur (#26).
- **Jeu. 19** : roster saisi (#23), feature freeze à 18 h.
- **Ven. 20** : v1.0.0, répétition de la soutenance, rétro finale.

9 Backlog : les 35 issues

Créez toutes les issues pendant le Sprint 0, dans cet ordre et avant la première PR (le script `scripts/create_backlog.py` le fait pour vous) : GitHub numérote issues et PR dans la même suite. Priorités : **P0** indispensable, **P1** saute en premier en cas de retard, **P2** bonus.

Sprint 0 · Conception · mar. 3 → mer. 4 nov. · A 5 pts / B 5 pts

Objectif : GDD validé, contrat d'API défini à deux, dépôt prêt. Aucune feature n'est codée avant la validation du GDD.

Issue	Titre	Dev	Pts	Prio	Dépend de
#1	`infra` Squelette du dépôt : projet Maven JavaFX qui ouvre une fenêtre, JUnit 5, packages	B	1	P0	—
#2	`docs` GDD · vision, boucle de jeu, maquettes des écrans (dont le dashboard), direction artistique et sonore	B	3	P0	—
#3	`docs` GDD · règles de combat, 10 types, formule de dégâts, talents, objets, liste des 30 fakemon	A	3	P0	—
#4	`core` Contrat d'API : PlayerAction, BattleEvent, BattleListener, BattleEngine, AiController, SpeciesRepository	A	2	P0	#2, #3
#5	`docs` UML v1 · cas d'utilisation et navigation entre les écrans	B	1	P0	#2

Sprint 1 · Fondations · jeu. 5 → mar. 10 nov. · A 11 pts / B 11 pts

Objectif : Modèle, types et dégâts testés ; la CI tourne ; menu, aperçu, combat et fin s'affichent sur des données factices.

Issue	Titre	Dev	Pts	Prio	Dépend de
#6	`docs` UML v1 · diagramme de classes du cœur + séquence d'un tour	A	1	P0	#4
#7	`infra` CI GitHub Actions : build et tests à chaque PR, rapport de couverture JaCoCo	A	2	P0	#1
#8	`core` Modèle : Type, Stats, Species, Fakemon, Move, Team	A	3	P0	#4
#9	`core` Table des types + calcul des dégâts (STAB, critique, précision, aléatoire injectable)	A	3	P0	#8
#10	`core` Générateur d'équipes : 2 × 6 tirés dans le roster, 4 attaques tirées, seed	A	1	P0	#8
#11	`ai` IA aléatoire : une attaque au hasard parmi les coups possibles	A	1	P0	#4, #8
#12	`ui` SceneManager + menu principal	B	2	P0	#1
#13	`ui` Écran de combat sur données factices : sprites, barres de PV, 4 attaques, journal	B	5	P0	#4, #12
#14	`ui` Écran d'aperçu des équipes (6 contre 6)	B	1	P0	#12
#15	`ui` Écran de fin de combat + revanche	B	1	P0	#12
#16	`art` Charte graphique + sprites lot 1 : 15 fakemon de face (32×32) + icônes des 10 types	B	2	P0	#3

Sprint 2 · Moteur & dashboard · jeu. 12 → lun. 16 nov. · A 8 pts / B 8 pts

Objectif : Le moteur résout un combat complet en tests (switchs, KO, victoire) ; le dashboard permet de saisir un fakemon ; les 30 sprites existent.

Issue	Titre	Dev	Pts	Prio	Dépend de
#17	`core` Moteur de tour : playTurn, priorité puis vitesse, KO, victoire et défaite	A	5	P0	#8, #9
#18	`core` Switch volontaire et remplacement forcé après KO	A	3	P0	#17
#19	`ui` Panneau d'équipe : switch et choix forcé après KO	B	2	P0	#13
#20	`ui` Dashboard : nom, nom FR, sprites avant/arrière, cri, attaques disponibles, types, stats	B	5	P0	#4, #12
#21	`art` Sprites lot 2 : 15 fakemon de face	B	1	P0	#16

Sprint 3 · Intégration & release · mar. 17 → ven. 20 nov. · A 12 pts / B 12 pts

Objectif : Un combat 6 contre 6 jouable contre l'IA sur le vrai roster, avec talents, objets, animations et sons ; documentation et soutenance prêtes ; v1.0.0 vendredi 20.

Issue	Titre	Dev	Pts	Prio	Dépend de
#22	`data` SpeciesRepository JSON : lecture/écriture, Factory, validation, dossier utilisateur	A	5	P0	#4, #8
#23	`data` Roster : 30 fakemon + ~30 attaques, saisis via le dashboard	A	2	P0	#20, #22
#24	`core` Talents (4) et objets tenus (3), objets en Decorator	A	3	P1	#17
#25	`docs` Javadoc du cœur + UML final + couverture ≥ 70 %	A	2	P0	—
#26	`ui` Branchement de l'écran de combat sur le vrai moteur (BattleListener, IA, switchs)	B	3	P0	#11, #17, #18, #19
#27	`anim` Animations : impact, PV qui descendent, KO, entrée et sortie	B	2	P1	#26
#28	`audio` Musiques, effets sonores et cris des fakemon (jsfxr), volume	B	2	P1	#12
#29	`ui` Bannières et icônes des talents et objets	B	1	P1	#24, #26
#30	`docs` README, Javadoc des écrans, UML final des écrans	B	2	P0	—
#31	`docs` Support de soutenance + script de démo	B	2	P0	—

Bonus · si temps libéré · après le Go / No-Go du lun. 16 nov. · A 5 pts / B 6 pts

Règle : Démarré seulement si tout le P0 des Sprints 1 et 2 est mergé. Ne s'ajoute jamais à la charge : utilise le temps libéré, ou remplace des P1 sur décision du PO.

Issue	Titre	Dev	Pts	Prio	Dépend de
#32	`core` Statuts : brûlure, poison, paralysie (pattern State)	A	3	P2	#17
#33	`ai` IA heuristique : choisit l'attaque aux dégâts attendus les plus élevés	A	2	P2	#11, #17
#34	`net` Spike réseau (timebox 2 h) : deux programmes échangent un event par socket	B	1	P2	—
#35	`net` Combat en LAN : hôte, invité, lobby (timebox 10 h)	B	5	P2	#26, #34

10 Workflow Git et issues

Règles de `main`

- Branche protégée : pas de push direct, PR obligatoire.
- CI verte et approbation de l'autre dev exigées pour merger.
- Squash-merge, branche supprimée après merge.
- `core.api` en CODEOWNERS A + B.
- Un tag par fin de sprint : v0.1.0, v0.2.0, v1.0.0.

Branches et commits

- Une issue = une branche = une PR.
- Nommage : `feat/17-moteur-de-tour`, `fix/…`, `docs/…`, `chore/…`
- Conventional Commits : `feat(engine): priorité puis vitesse (#17)`
- Rebase quotidien sur `main`, PR de moins de 400 lignes.

Flux de travail

- Board GitHub Projects : Backlog → Sprint → En cours → En revue → Terminé.
- Deux issues « En cours » au maximum par dev.
- Relecture sous 24 h ouvrées : relire passe avant d'ouvrir une nouvelle issue.
- Description de PR : quoi, pourquoi, comment tester, captures pour l'UI.

Labels et milestones

- Domaine : `core` `ai` `data` `ui` `anim` `audio` `art` `net` `infra` `docs` `test` `bug` `retro-action`
- Responsable : `dev:A` `dev:B` · taille : `sp:1` à `sp:5` · priorité : `P0` `P1` `P2`
- Un milestone par sprint, plus un milestone « Bonus ».

Definition of Ready

- Story formulée : « En tant que…, je veux…, afin de… »
- Critères d'acceptation testables
- Estimation en points
- Dépendances levées, responsable assigné

Definition of Done

- Branche dédiée, PR liée à l'issue
- Tests de la logique ajoutée ; CI verte
- Javadoc sur les classes et méthodes publiques
- PR approuvée par l'autre dev, squash-merge sur `main` avec « Closes #n »
- Démontrable depuis `main` ; UML mis à jour si l'architecture a changé

Exemple d'issue (générée par le script)

```
Titre : [CORE] Moteur de tour : playTurn, priorité puis vitesse, KO, victoire et défaite

Sprint : Sprint 2 · Moteur & dashboard · Responsable : Dev A · Points : 5 (≈ 10 h) · Priorité : P0
Dépend de : #8, #9

En tant que joueur, je veux que les actions des deux camps se résolvent dans un ordre juste,
afin que la vitesse et la priorité des attaques comptent vraiment.

Critères d'acceptation
- [ ] playTurn(actionJoueur, actionIA) résout le tour complet
- [ ] Priorité d'abord, puis vitesse ; en cas d'égalité, tirage avec la graine
- [ ] Un fakemon mis KO avant son tour n'agit pas
- [ ] Chaque étape envoie un BattleEvent aux BattleListener, dans l'ordre
- [ ] BattleEnded est émis quand une équipe n'a plus de fakemon apte ; tests pour chaque critère
```

```
Exemple de PR : feat(engine): playTurn par priorité puis vitesse (#17)

Quoi       : BattleEngine ordonne les actions (priorité, vitesse, graine) et notifie les listeners
Pourquoi   : Closes #17
Tester     : mvn test -Dtest=BattleEngineTest ; 8 cas dont égalité de vitesse et KO avant d'agir
Relecteur  : Dev B
```

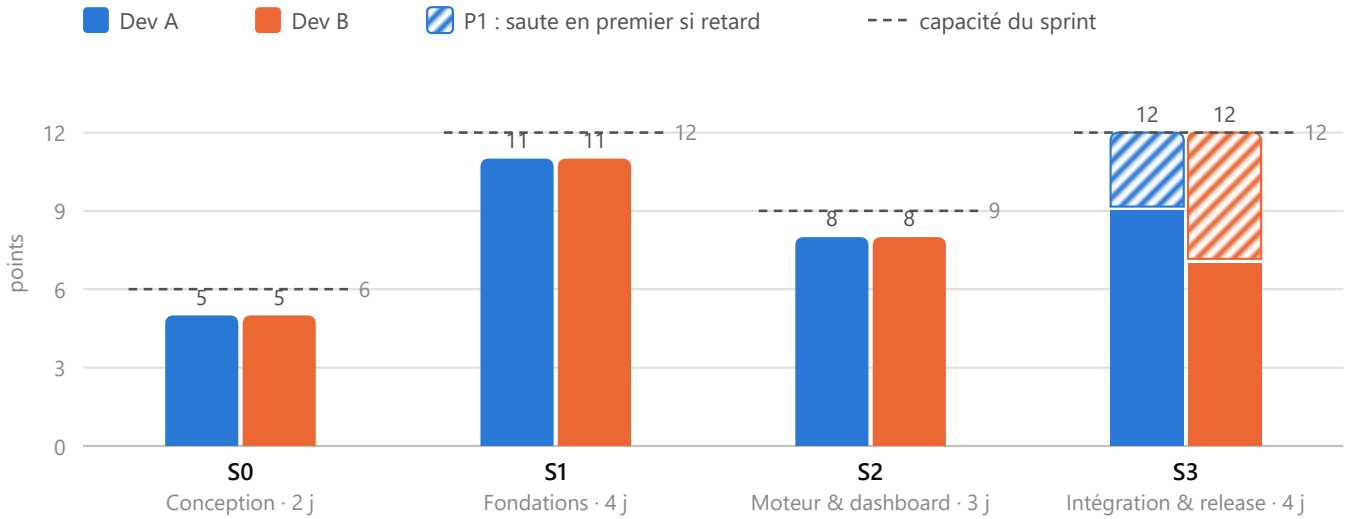
11 Charge de travail

Dev A
36 pts
33 P0 + 3 P1 · bonus possible +5

Dev B
36 pts
31 P0 + 5 P1 · bonus possible +6

Capacité par dev
39 pts
13 jours ouvrés × 3 points (≈ 78 h)

Points planifiés par sprint et par dev, face à la capacité du sprint



A a un peu plus de P0 (le moteur), B un peu plus de P1 (animations, sons, bannières). Le Sprint 3 est le seul rempli jusqu'à la capacité, et seulement grâce aux P1 : en cas de retard, ce sont eux qui sautent.

12 Risques et parades

Risque	Proba.	Impact	Parade	Suivi
Les technos nouvelles (JavaFX, Maven, Gson) ralentissent	Haute	Moyen	Apprentissage compté dans les estimations, bootstrap JavaFX fourni, entraide signalée au daily, P1 sacrifiables.	A B
Intégration écran ↔ moteur le mardi 17	Moyenne	Haut	Contrat défini à deux le 3 (#4), écrans construits sur les vraies classes d'events, intégration dès le 1er jour du S3.	A B
30 sprites : gros volume d'assets	Moyenne	Moyen	32×32, palette réduite, 20 min maximum par sprite, dos = face retournée, placeholders colorés par type.	B
Roster saisi tard (jeudi 19)	Moyenne	Moyen	Repli : JSON écrit à la main, relu tel quel par le dashboard ; données de test dès le S1.	A
Le LAN dévore le temps	Faible	Haut	Bonus, Go / No-Go, branche à part, timebox de 10 h, jamais ajouté à la charge.	B
Le dashboard ne peut pas écrire dans le jar	Certaine	Moyen	Dossier utilisateur fusionné au roster au chargement (#22).	A

Risque	Proba.	Impact	Parade	Suivi
Un dev indisponible quelques jours	Faible	Haut	Relecture croisée : chacun connaît le code de l'autre ; les P1 de l'absent sautent.	A B
Conflits Git	Moyenne	Faible	Packages séparés par propriétaire, PR courtes, rebase quotidien.	A B

13 Soutenance et livrables

Déroulé proposé (environ 20 min)

Séquence	Contenu	Durée	Qui
Pitch	Le jeu en une minute, les options choisies	2 min	B
GDD et UX	Boucle de jeu, maquettes, direction artistique, dashboard	3 min	B
Architecture	Contrat `core.api`, un tour de jeu sans thread	3 min	A
Patterns et UML	Observer, Factory, Decorator, Strategy, Template Method	4 min	A
Qualité	Tests, CI, couverture, Javadoc	2 min	A
Démo jouable	Voir le script ci-dessous	5 min	B pilote, A commente
Méthode	Burndown, rétros, ce qu'on a changé en route	2 min	A

Script de démo (#31)

1. Lancer le jeu avec une graine de démo pour un tirage reproductible.
2. Ouvrir le dashboard, créer un fakemon en direct (sprites, cri, attaques, nom, nom français), l'enregistrer.
3. Combat contre l'IA : montrer un super efficace, un talent qui se déclenche, un objet tenu, un switch forcé après KO.
4. Si le bonus LAN a été fait : quelques tours entre les deux PC.
5. Montrer une PR mergée, la CI verte, le rapport de couverture et la Javadoc.

Checklist de rendu

Livrable	Emplacement	Dev
Code source et tag `v1.0.0` (visé vendredi 20 à 17 h, dernier délai dimanche 22 au soir), toutes les issues P0 fermées	`main`	B
Game Design Document final	`docs/GDD.md`	A B
Diagrammes UML (sources PlantUML et PNG)	`docs/uml/`	A B
Rapport de couverture JaCoCo	Artefact CI	A
Javadoc	Artefact CI	A B
README avec captures et crédits	`README.md`	B
Support de soutenance et script de démo	`docs/soutenance/`	B
Comptes rendus des rétros et burndown	`docs/scrum/`	A